How Group Theory Secures Online Banking and E-Commerce: A Study of RSA Cryptography

M1: Geetika Pachauri (SE24UCAM070)
M2: Anusha Anindita (SE24UCAM006)
M3: Harshil Pansala (SE24UCAM051)
M4: Shirsha Pattanaik (SE24UCAM027)
M5: Pearl Mendapara (SE24UCAM043)

Submitted to:
Professor Pradeep Kumar
Department of Mathematics
Mahindra University

November 2025

Abstract

This report explores the mathematical foundations of RSA cryptography through the lens of group theory and demonstrates its critical role in securing modern online banking and e-commerce systems. We present the algebraic structures underlying RSA, provide a complete mathematical proof of correctness using Euler's theorem, demonstrate the algorithm through worked examples, analyze its security properties, and examine real-world applications. The report also compares RSA with elliptic curve cryptography and discusses future challenges posed by quantum computing.

Contents

1 Introduction **5**

2 Mathematical Foundations: Group Theory **5**
2.1 Definition of a Group . . . 5
2.2 The Multiplicative Group $\mathbb{Z}_n^*$ . . . 5
2.2.1 Properties . . . 6
2.3 Example: $\mathbb{Z}_{10}^*$ . . . 6
2.4 Euler's Totient Function . . . 6
2.5 Euler's Theorem as a Group-Theoretic Result . . . 6

3 RSA Algorithm and Group Structure **7**
3.1 Key Generation . . . 7
3.2 Prime Numbers in RSA . . . 7
3.3 Encryption (in Group Terms) . . . 7
3.4 Decryption . . . 8

4 Correctness Proof Using Group Theory **8**

5 Complete Worked Example **8**
5.1 Step A – Pick Primes . . . 8
5.2 Step B – Compute Modulus . . . 8
5.3 Step C – Compute Totient (Group Order) . . . 8
5.4 Step D – Select Public Exponent . . . 8
5.5 Step E – Compute Private Exponent d . . . 9
5.6 Step F – Public & Private Keys . . . 9
5.7 Step G – Encrypt a Message . . . 9
5.8 Step H – Decrypt the Ciphertext . . . 10

6 Security Analysis Through Group Theory **10**
6.1 The Underlying Group Structure . . . 10
6.2 Encryption and Decryption as Group Exponentiation . . . 10
6.3 Why the Inverse Exponent Cannot Be Computed . . . 10
6.4 Role of Group Size . . . 10
6.5 Summary of Group-Theoretic Security . . . 10

7 Real-World Applications **11**
7.1 RSA in Online Banking . . . 11
7.1.1 Account Authentication . . . 11
7.1.2 Transaction Security . . . 11
7.1.3 ATM Communications . . . 11
7.1.4 Compliance Standards . . . 11
7.2 RSA in E-Commerce . . . 11
7.2.1 Payment Processing . . . 11
7.2.2 Credit Card Protection . . . 12
7.2.3 Payment Gateway Integration . . . 12
7.2.4 PCI DSS Requirements . . . 12
7.3 RSA in HTTPS, SSL, and TLS . . . 12

7.3.1 TLS Handshake Using RSA 12
7.3.2 Hybrid Encryption Approach 12
7.3.3 Certificate Authorities and Trust Model 12
7.4 Digital Signatures with RSA 12
7.4.1 How Signing Works 13
7.4.2 Real-World Applications 13

8 Efficiency Analysis: RSA vs ECC 13
8.1 Comparison Table 13
8.2 Strengths and Weaknesses 13
8.3 Recommendations 14

9 Future Directions and Quantum Computing 14
9.1 Quantum Computing Threat 14
9.2 Post-Quantum Cryptography 14
9.3 Current Recommendations 14

10 Conclusion 14

11 Python Implementation 15

1 Introduction

Apart from being a highly interesting and beautiful subject, group theory has many applications and is widely used across various fields. In this report, we focus specifically on its applications in cryptography, particularly RSA encryption.

RSA (Rivest–Shamir–Adleman) is a public-key cryptosystem whose operations take place inside the multiplicative group of units modulo n, denoted $(Z_n^*, \times)$. The algorithm uses properties of this finite group, especially Euler's theorem, and the computational difficulty of factoring n to provide secure encryption and digital signatures.

RSA is fundamental to modern internet security, forming the backbone of secure communications in online banking, e-commerce, and digital signatures. When you see the padlock icon in your browser during an HTTPS connection, RSA (or its elliptic curve variants) is often working behind the scenes to establish a secure channel.

2 Mathematical Foundations: Group Theory

2.1 Definition of a Group

A group is a mathematical structure consisting of a set G together with a binary operation $*$ that combines any two elements of the set to produce another element in the same set, such that the following four properties are satisfied:

1. **Closure**: For all $a, b \in G$,
$$a * b \in G$$

2. **Associativity**: For all $a, b, c \in G$,
$$(a * b) * c = a * (b * c)$$

3. **Identity Element**: There exists an element $e \in G$ such that for every $a \in G$,
$$a * e = e * a = a$$

4. **Inverse Element**: For every $a \in G$, there exists an element $a^{-1} \in G$ such that
$$a * a^{-1} = a^{-1} * a = e$$

2.2 The Multiplicative Group $\mathbb{Z}_n^*$

The set $\mathbb{Z}_n^*$ is defined as:
$$\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n \mid \gcd(a, n) = 1\}$$

where $\mathbb{Z}_n^*$ consists of all integers modulo n that are relatively prime to n.

Under the operation of **multiplication modulo** n, the set $\mathbb{Z}_n^*$ forms a **group**, called the multiplicative group modulo n.

2.2.1 Properties

1. **Closure**: If $a, b \in \mathbb{Z}_n^*$, then $ab \bmod n \in \mathbb{Z}_n^*$.
2. **Associativity**: Multiplication modulo n is associative.
3. **Identity**: The identity element is 1, since
$$a \times 1 \equiv a \pmod{n}$$
4. **Inverse**: Every element $a \in \mathbb{Z}_n^*$ has a multiplicative inverse a^{-1} such that
$$a \times a^{-1} \equiv 1 \pmod{n}$$

2.3 Example: $\mathbb{Z}_{10}^*$

Let $n = 10$. Then the group is:
$$\mathbb{Z}_{10}^* = \{1, 3, 7, 9\}$$

Note that the elements included in the set are coprime to n.

Identity example:
$$3 \times 1 \equiv 3 \pmod{10}$$

Closure example:
$$3 \times 7 = 21 \equiv 1 \pmod{10}$$

Inverse example:
$$3 \times 7 \equiv 1 \pmod{10}$$

So **7 is the inverse of 3**.

2.4 Euler's Totient Function

For a prime p, $\varphi(p) = p - 1$. Therefore, when $n = pq$ with p and q prime, the totient is:
$$\varphi(n) = (p-1)(q-1)$$

The totient function is essential because when $n = pq$, the set $\mathbb{Z}_n^* = \{a : \gcd(a, n) = 1\}$ forms a multiplicative group modulo n with size $\varphi(n)$. This is the order of the group.

2.5 Euler's Theorem as a Group-Theoretic Result

Euler's Theorem states: $a^{\varphi(n)} \equiv 1 \pmod{n}$ for all $a \in \mathbb{Z}_n^*$.

This follows from group theory:

- $(\mathbb{Z}_n^*, \times)$ is a finite group
- Its order is $\varphi(n)$
- Lagrange's Theorem states that $a^{|G|} =$ identity element

Thus $a^{\varphi(n)} \equiv 1 \pmod{n}$.

Since $(\mathbb{Z}_n^*, \times)$ is a finite group of order $\varphi(n)$, Lagrange's theorem directly gives the identity result $a^{\varphi(n)} = 1 \bmod n$. This group-theoretic "return to identity" is the fundamental reason RSA decryption correctly recovers the message.

3 RSA Algorithm and Group Structure

3.1 Key Generation

Step 1: Choose two distinct large primes p and q

Group comment: Primes are chosen so that $\mathbb{Z}_n^*$ behaves predictably; $n = pq$ will be our modulus.

Step 2: Compute $n = p \cdot q$

The encryption group will be (a subgroup of) $\mathbb{Z}_n^*$.

Step 3: Compute $\varphi(n) = (p-1)(q-1)$

This equals the order of $\mathbb{Z}_n^*$ when p, q are prime. This is the group order we'll use for exponent arithmetic.

Step 4: Choose public exponent e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$

This ensures e has a multiplicative inverse modulo $\varphi(n)$.

Step 5: Compute d such that $e \cdot d \equiv 1 \pmod{\varphi(n)}$

Then d is the multiplicative inverse of e in $(\mathbb{Z}_{\varphi(n)}^*, \times)$. We use the Extended Euclidean Algorithm for this.

Step 6: Keys:

- **Public key:** (e, n) – shared openly
- **Private key:** (d, n) – kept secret

3.2 Prime Numbers in RSA

RSA uses two large prime numbers to generate the public key because the security of the system depends on the computational difficulty of factorizing their product back into the original primes. If we choose $n = pq$, where p and q are large primes, then n can be publicly shared without revealing p or q. For sufficiently large primes, no known classical algorithm can factor n efficiently. This property ensures that only the legitimate owner—who knows p and q—can compute the private key.

The hardness of integer factorization stems from the fact that the running time of the best-known factorization algorithms depends on the size of the second-largest prime factor. Thus, even when the public exponent e is known, an attacker cannot compute the private exponent d without factoring n.

The security also relies on:

- The size of the primes
- The randomness of the primes

Randomness generation is a challenge for classical computers. Various organizations employ entropy sources to generate truly unpredictable primes, such as the lava lamp randomness wall at Cloudflare.

3.3 Encryption (in Group Terms)

To send message $m \in \mathbb{Z}_n^*$, compute:

$$c \equiv m^e \pmod{n}$$

This is exponentiation in $\mathbb{Z}_n^*$. Anyone with the public key can encrypt.

3.4 Decryption

$$m \equiv c^d \equiv (m^e)^d \equiv m^{ed} \pmod{n}$$

Only someone with the private key d can decrypt.

4 Correctness Proof Using Group Theory

Because $ed \equiv 1 \pmod{\varphi(n)}$, there exists integer k such that:

$$ed = 1 + k\varphi(n)$$

For $m \in \mathbb{Z}_n^*$, we have:

$$\begin{aligned} m^{ed} &= m^{1+k\varphi(n)} \\ &= m \cdot (m^{\varphi(n)})^k \\ &\equiv m \cdot 1^k \\ &\equiv m \pmod{n} \end{aligned}$$

The critical step uses Euler's theorem: $m^{\varphi(n)} \equiv 1 \pmod{n}$ for any $m \in \mathbb{Z}_n^*$.

Therefore: Decryption perfectly recovers the original message. The operations are inverses in the group structure. This demonstrates how the properties of groups help keep RSA computations consistent, and modular inverses are used in calculating the private key.

5 Complete Worked Example

Warning: We use small primes for demonstration only. In real cryptographic systems, primes must be at least 1024 bits (typically 2048 or 4096 bits) to ensure security.

5.1 Step A – Pick Primes

$$p = 61, \quad q = 53$$

5.2 Step B – Compute Modulus

$$n = p \cdot q = 61 \times 53 = 3233$$

5.3 Step C – Compute Totient (Group Order)

$$\varphi(n) = (p-1)(q-1) = 60 \times 52 = 3120$$

This is the order of the multiplicative group $\mathbb{Z}_{3233}^*$.

5.4 Step D – Select Public Exponent

Choose $e = 17$

Verify: $\gcd(17, 3120) = 1$ ✓

5.5 Step E – Compute Private Exponent d

Find d such that $17d \equiv 1 \pmod{3120}$

Using the Extended Euclidean Algorithm:

Step	Remainder	Quotient	Coefficients
Init	3120, 17	—	$s = 0, 1$; $t = 1, 0$
1	9	183	$s = -183$; $t = 1$
2	8	1	$s = 184$; $t = -1$
3	1	1	$s = -367$; $t = 2$
4	0	8	gcd = 1

Result: $17(-367) + 3120(2) = 1$

Therefore:

$$d \equiv -367 \equiv 3120 - 367 = 2753 \pmod{3120}$$

Verification: $17 \times 2753 = 46{,}801 = 15 \times 3120 + 1$ ✓

5.6 Step F – Public & Private Keys

- **Public key:** $(e, n) = (17, 3233)$
- **Private key:** $(d, n) = (2753, 3233)$

5.7 Step G – Encrypt a Message

Let message $m = 65$

(Verify: $1 \leq 65 < 3233$ and $\gcd(65, 3233) = 1$ ✓)

Compute:

$$c \equiv m^e \pmod{n} = 65^{17} \bmod 3233$$

Using Square-and-Multiply Method:

Power	Computation	Result (mod 3233)
65^1	65	65
65^2	$65^2 = 4{,}225$	992
65^4	$992^2 = 984{,}064$	1,232
65^8	$1{,}232^2 = 1{,}518{,}824$	1,547
65^{16}	$1{,}547^2 = 2{,}393{,}209$	789

Since $17 = 16 + 1$:

$$\begin{aligned} 65^{17} &\equiv 65^{16} \cdot 65 \pmod{3233} \\ &\equiv 789 \times 65 \pmod{3233} \\ &\equiv 51{,}285 \pmod{3233} \\ &\equiv 2790 \pmod{3233} \end{aligned}$$

Ciphertext: $c = 2790$

5.8 Step H – Decrypt the Ciphertext

Compute:

$$m \equiv c^d \bmod n = 2790^{2753} \bmod 3233$$

Using fast modular exponentiation:

Result: $m = 65$ ✓

Conclusion: Encryption and decryption are perfect inverses, as the group-theoretic proof predicted.

6 Security Analysis Through Group Theory

6.1 The Underlying Group Structure

RSA operates inside the multiplicative group $(\mathbb{Z}_n^*, \times)$, where $\mathbb{Z}_n^*$ is the set of integers relatively prime to n. This forms a finite abelian group under multiplication modulo n. The size of this group is determined by Euler's totient function $\varphi(n) = (p-1)(q-1)$, where $n = pq$ for large primes p and q. This group order is known only to the party who knows p and q.

6.2 Encryption and Decryption as Group Exponentiation

RSA encryption is defined by $C = M^e \bmod n$, where e is the public exponent. Decryption is the inverse transformation $M = C^d \bmod n$, where d is the private exponent satisfying $ed \equiv 1 \pmod{\varphi(n)}$. Thus, decryption is the inverse group operation of encryption.

6.3 Why the Inverse Exponent Cannot Be Computed

An attacker attempting to determine the private key d from the public key (e, n) must compute the inverse of e modulo $\varphi(n)$. However, this requires computing $\varphi(n)$, which in turn requires knowing p and q.

Hard Problem 1: Factoring n

Finding p and q from $n = pq$ is the integer factorization problem, which has no known efficient solution for large n.

Hard Problem 2: Inverting Exponentiation in a Large Group

Breaking RSA also requires solving $M^e \equiv C \pmod{n}$, which is equivalent to extracting an e-th root modulo n. Without knowing d, this inversion is computationally infeasible.

6.4 Role of Group Size

The size of the group $|\mathbb{Z}_n^*| = \varphi(n)$ is extremely large for a 2048-bit modulus, making brute-force attempts or root extraction infeasible.

6.5 Summary of Group-Theoretic Security

RSA is secure because:

- Encryption and decryption are inverse exponentiations in $(\mathbb{Z}_n^*, \times)$

- The inverse exponent d cannot be computed without the group order $\varphi(n)$
- Determining $\varphi(n)$ requires factoring $n = pq$, which is computationally difficult
- Reversing exponentiation (extracting discrete roots modulo n) is infeasible without the private key

Thus, RSA's security relies directly on group theory concepts: group inverses, exponentiation in finite abelian groups, and the protection offered by a hidden group order.

7 Real-World Applications

7.1 RSA in Online Banking

Online banking systems depend heavily on RSA to protect customer information, secure transactions, and authenticate users.

7.1.1 Account Authentication

RSA ensures safe transmission of login credentials and OTP information. Data is encrypted using the bank's public key and can only be decrypted by the bank's private key.

7.1.2 Transaction Security

RSA secures financial transactions, including fund transfers, account changes, and verification procedures. A hybrid approach is commonly used: RSA is used to exchange symmetric keys, and AES encrypts the actual transaction data.

7.1.3 ATM Communications

ATM machines rely on RSA certificates to authenticate themselves to the bank's server. RSA prevents counterfeit ATMs and secures PIN transmission through encrypted key exchange.

7.1.4 Compliance Standards

Banks use RSA to comply with global security regulations such as PCI DSS, RBI Cybersecurity Framework, and ISO 27001.

7.2 RSA in E-Commerce

RSA is fundamental to securing online shopping and digital payment processes.

7.2.1 Payment Processing

RSA protects sensitive information such as user identity, order information, and payment tokens during online transactions.

7.2.2 Credit Card Protection

During transactions, RSA encryption ensures that credit card data remains confidential when transmitted between the customer, merchant, and payment gateway.

7.2.3 Payment Gateway Integration

RSA-based TLS encryption is used in systems such as PayPal, Stripe, Razorpay, and banking merchant services to maintain confidentiality and prevent fraud.

7.2.4 PCI DSS Requirements

RSA supports PCI DSS compliance by providing strong cryptographic protection for cardholder data during transmission and authentication.

7.3 RSA in HTTPS, SSL, and TLS

RSA is a cornerstone of secure internet communication.

7.3.1 TLS Handshake Using RSA

1. Client sends a "Hello" message with supported cipher suites
2. Server responds with its RSA certificate
3. Client generates a session key and encrypts it with the server's public key
4. Server decrypts the session key using its private key
5. Both parties switch to symmetric encryption (AES) for faster communication

7.3.2 Hybrid Encryption Approach

RSA is used for secure key exchange, while AES is used for bulk encryption. HMAC ensures message integrity. This combination provides both security and performance efficiency.

7.3.3 Certificate Authorities and Trust Model

Certificate Authorities (CAs) issue RSA-based certificates. Browsers verify certificate validity, ensuring protection from spoofed or fraudulent websites.

7.4 Digital Signatures with RSA

RSA digital signatures provide authentication, integrity, and non-repudiation.

7.4.1 How Signing Works

1. Sender creates a hash of the message
2. The hash is encrypted using the sender's private key, forming the digital signature
3. Receiver decrypts the signature using the sender's public key
4. Receiver recomputes and compares the hash to verify authenticity

7.4.2 Real-World Applications

- Software signing (e.g., Windows executables)
- Email authentication (S/MIME)
- Legal document verification
- Blockchain validation mechanisms

8 Efficiency Analysis: RSA vs ECC

8.1 Comparison Table

Feature	**RSA**	**ECC**
Key Size	2048–4096 bits	256–521 bits
Security per Bit	Lower	Higher
Speed	Slower	Faster
Computation Cost	High	Low
Mobile/IoT Usage	Less efficient	Highly efficient
Use Cases	Banking certificates, legacy systems	Modern TLS, mobile apps

8.2 Strengths and Weaknesses

RSA Strengths:

- Widely adopted and trusted
- Easy to implement
- Strong for digital signature workflows

RSA Weaknesses:

- Large key sizes
- Slow key generation and decryption
- Not suitable for constrained devices

ECC Strengths:

- High security with small key sizes

- Fast computations
- Ideal for mobile and IoT applications

ECC Weaknesses:

- Mathematically complex
- Past patent restrictions

8.3 Recommendations

- RSA-2048 remains secure and widely compatible
- ECC (e.g., P-256) is recommended for new systems
- Hybrid RSA–ECC systems provide strong multi-device compatibility

9 Future Directions and Quantum Computing

9.1 Quantum Computing Threat

The emergence of quantum computing poses a significant threat to RSA security. Shor's algorithm can factor large numbers efficiently on quantum computers, potentially breaking RSA encryption within decades.

9.2 Post-Quantum Cryptography

NIST is standardizing post-quantum cryptographic algorithms such as CRYSTALS-Kyber and CRYSTALS-Dilithium to replace RSA in quantum-vulnerable applications.

9.3 Current Recommendations

- Use RSA-3072 or RSA-4096 for new deployments requiring long-term security
- Consider ECC for resource-constrained environments
- Plan migration strategies toward post-quantum cryptography
- Implement crypto-agility to support future algorithm changes

10 Conclusion

This report has demonstrated how group theory provides the mathematical foundation for RSA cryptography, one of the most important security technologies in use today. By operating within the multiplicative group $\mathbb{Z}_n^*$ and leveraging Euler's theorem, RSA achieves secure encryption through the computational difficulty of factoring large composite numbers.

RSA remains the backbone of digital security, protecting trillions of dollars in transactions daily. Its applications span online banking, e-commerce, secure communications,

and digital signatures. While more efficient alternatives like ECC are emerging and post-quantum cryptography looms on the horizon, RSA's proven security and widespread deployment ensure its continued relevance.

The group-theoretic perspective reveals why RSA works: encryption and decryption are inverse operations in a finite abelian group, and the security rests on the hidden group order $\varphi(n)$, which cannot be computed without factoring n. This elegant connection between abstract algebra and practical cryptography demonstrates the power of pure mathematics in solving real-world problems.

Organizations must balance immediate security needs with long-term sustainability by adopting crypto-agile architectures and planning for the post-quantum era. As technology evolves, the mathematical principles underlying RSA—rooted in group theory—will continue to inform the development of next-generation cryptographic systems.

11 Python Implementation

```python
# rsa_demo.py - RSA demonstration (educational only)

def egcd(a, b):
    """Extended Euclidean Algorithm"""
    if b == 0:
        return (1, 0, a)
    x, y, g = egcd(b, a % b)
    return (y, x - (a // b) * y, g)

def modinv(a, m):
    """Compute modular inverse"""
    x, y, g = egcd(a, m)
    if g != 1:
        raise Exception('No inverse')
    return x % m

def validate_message(m, n):
    """Validate message in Z*_n"""
    if not (1 <= m < n):
        raise ValueError("Invalid range")
    if egcd(m, n)[2] != 1:
        raise ValueError("gcd must be 1")

# Example parameters (SMALL - demo only!)
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17
d = modinv(e, phi)

print("RSA KEY GENERATION")
print(f"Prime p: {p}")
print(f"Prime q: {q}")
```

```python
print(f"Modulus n: {n}")
print(f"phi(n): {phi}")
print(f"Public exponent e: {e}")
print(f"Private exponent d: {d}")
print(f"Public Key: ({e}, {n})")
print(f"Private Key: ({d}, {n})")

# ENCRYPTION
m = 65
validate_message(m, n)
c = pow(m, e, n)
print(f"\nEncrypting m={m}")
print(f"Ciphertext c: {c}")

# DECRYPTION
m_dec = pow(c, d, n)
print(f"\nDecrypting c={c}")
print(f"Recovered m: {m_dec}")

# Verification
if m == m_dec:
    print("SUCCESS: Decryption works!")
else:
    print("FAILURE")
```


Acknowledgments

We would like to express our sincere gratitude to Professor Pradeep Kumar for his guidance and support throughout this project. His expertise in mathematics and encouragement helped us explore the beautiful connection between abstract algebra and practical cryptography.

References

[1] Rivest, R.L., Shamir, A., & Adleman, L. (1978). *A method for obtaining digital signatures and public-key cryptosystems*. Communications of the ACM, 21(2), 120-126.

[2] Katz, J., & Lindell, Y. (2014). *Introduction to Modern Cryptography* (2nd ed.). CRC Press.

[3] Dummit, D.S., & Foote, R.M. (2004). *Abstract Algebra* (3rd ed.). John Wiley & Sons.

[4] National Institute of Standards and Technology (NIST). *Post-Quantum Cryptography Standardization*. Retrieved from `https://csrc.nist.gov/projects/post-quantum-cryptography`

[5] Cloudflare. (2017). *Randomness 101: LavaRand in Production*. Retrieved from `https://blog.cloudflare.com/randomness-101-lavarand-in-production/`